

Product Inventory Management System

1. Introduction

A Product Inventory Management System is a software application used to track, manage, and organize products in a store or company. It helps businesses monitor stock levels, sales, purchases, and product details efficiently.

Inventory management ensures that the right products are available at the right time without overstocking or running out of items.

Team ID : SWTID-2026-6547

Team Size : 4

Team Leader : AMRIN BEGUM L

Team member : BHARATHI K

Team member : DHARSHINI PRIYA K

Team member : DHARSHINI T

2. Objectives

- To maintain accurate records of products.
- To track the quantity of items available in stock.
- To manage product purchases and sales.
- To reduce manual work and human errors.
- To improve business efficiency and decision making.

3. Features of Inventory Management System

- Product Management – Add, update, and delete product details.
- Stock Tracking – Monitor the quantity of each product in stock.
- Sales Management – Record product sales and update stock automatically.
- Purchase Management – Manage purchases from suppliers.
- Report Generation – Generate reports for stock, sales, and purchases.

4. Components of Inventory System

4.1 Product

Stores product information such as:

- Product ID
- Product Name
- Category
- Price
- Quantity

4.2 Supplier

Information about the suppliers who provide products.

4.3 Stock

Shows the available quantity of each product in the inventory.

4.4 Customer

Stores customer information when products are sold.

5. Advantages

- Saves time and effort.
- Reduces human errors.
- Helps in better stock control.
- Improves business productivity.
- Provides accurate reports.

6. Disadvantages

- Initial setup cost.
- Requires technical knowledge.
- System failure may affect operations.

7. Applications

- Inventory management systems are used in:
- Retail stores
- Supermarkets
- Warehouses
- Manufacturing industries
- E-commerce companies

Inventory product management

Ideation Phase

Brainstorm & Idea Prioritization Template

Brainstorming is used to generate creative ideas and identify the most suitable solution for a real-world problem. During the ideation phase, team members collaborated to discuss common issues faced in managing product inventories and explored multiple web-based solutions using the MERN stack. Ideas were evaluated based on feasibility, scalability, security, and learning outcomes.

Step-1: Team Gathering, Collaboration and

Select the Problem Statement

The team conducted a discussion session to identify common problems that could be solved using full-stack development. Several ideas were proposed such as:

- E-commerce product tracker

- Expense manager

- Task management system

- Cryptocurrency dashboard

- Inventory management system

After evaluating complexity, real-world usefulness, and alignment with MERN stack learning objectives, the team finalized the following problem statement:

Problem Statement:

Small businesses and individual users lack a simple, secure, and centralized system to

Manage their product inventory. Many existing solutions are complex or require paid subscriptions. Therefore, the team decided to build a JWT-based Product Inventory

Management System that allows users to securely manage their own products.

Step-2: Brainstorm, Idea Listing and

Grouping

During brainstorming, all team members contributed ideas without restrictions. The ideas were then grouped into categories:

Authentication & Security Ideas

- JWT-based login system
- User-specific product access
- Secure password hashing
- Role-based access (future scope)

Core Inventory Features

- Add product
- Update product details
- Delete product
- View all products
- Search products by name

User Experience Ideas

- Simple dashboard UI
- Responsive design
- Clean product cards layout
- Fast loading using React

Technical Ideas

- MERN stack architecture
- REST API development
- MongoDB database design
- Middleware for route protection

After grouping, the team focused on ideas that were realistic within the project timeline.

Step 3: Idea Prioritization

The team prioritized ideas based on:

- Feasibility within deadline
- Learning value
- Practical real-world use
- Technical complexity

High Priority (Implemented Features)

- JWT Authentication (Register/Login)
- Private product CRUD operations
- MongoDB database integration
- Secure API routes
- Search functionality

Medium Priority (Enhancements)

- Product filtering
- Improved UI components
- Error handling improvements

Low Priority / Future Scope

- Role-based admin system
- Analytics dashboard
- Notifications for low stock
- Deployment and cloud hosting

Final Decision:

The team finalized the Product Inventory Management System as it provides strong learning

opportunities in authentication, backend API design, and frontend integration using MERN stack

technologies

Project Design Phase-II

Solution Requirements (Functional & Non-functional)

Date

Team ID

Project Name

Maximum Marks

4 Marks

Functional Requirements:

Following are the functional requirements of the proposed solution.

FR No.

Functional Requirement (Epic)

Sub Requirement (Story / Sub-Task)

FR-1

User Authentication &

Authorization

User can register with email and password

User can log in securely using JWT

Only authenticated users can access dashboard

FR-2

Product Management

(CRUD)

Add new product with name, price, quantity and

Description

View all products in dashboard

Non-functional Requirements:

Following are the non-functional requirements of the proposed solution.

NFR

No.

Non-Functional Requirement

Description

NFR-1

Usability

The application should have a simple and
Intuitive user interface so users can manage
Products easily.

NFR-2

Security

JWT authentication must be used. Passwords
Should be hashed and unauthorized access must
Be prevented.

NFR-3

Reliability

The system should provide consistent
Performance with minimal server errors or
Downtime.

NFR-4

Performance

Product data should load quickly and API responses
Should be returned within 2 seconds.

NFR-5

Availability

The system should remain accessible to users with

Minimal downtime during operation.

NFR-6

Scalability

The architecture should support increasing numbers of users and products without performance issues.

Future features should be easy to integrate.

Product Inventory Management System Ideation Phase

Customer Problem Statement Template:

The Customer Problem Statement helps identify the challenges faced by users while managing

product inventories. Understanding the user's perspective allows the team to design a system

I am	Describe customer with 3-4 key characteristics - who are they?	Describe the customer and their attributes here
I'm trying to	List their outcome or "job" the care about - what are they trying to achieve?	List the thing they are trying to achieve here
but	Describe what problems or barriers stand in the way - what bothers them most?	Describe the problems or barriers that get in the way here
because	Enter the "root cause" of why the problem or barrier exists - what needs to be solved?	Describe the reason the problems or barriers exist
which makes me feel	Describe the emotions from the customer's point of view - how does it impact them emotionally?	Describe the emotions the result from experiencing the problems or barriers

that is secure, simple, and efficient. By analyzing user needs, the project focuses on creating a

I am	I'm trying to	But	Because	Which makes me feel
A small business owner	Manage and update product stock easily	Most inventory tools are complex or expensive	I need a simple system to quickly add and edit products	Frustrated managing inventory manually

Problem Statement	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	A small business owner	Manage product stock efficiently	Existing tools are complicated or expensive	I need a simple system to add and update products quickly	Frustrated managing inventory manually
PS-2	A freelancer selling products online	Track available items and pricing	Data gets lost without a centralized platform	I want a secure dashboard where only I can manage my products	Stressed about losing important product data
PS-3	A shop manager	Update stock quantity regularly	There is no easy real-time CRUD system	I need a fast and responsive web app to update inventory anytime	Anxious when stock information is outdated

PS-4	A new entrepreneur	Organize products in one place	Many inventory apps require complex setup	I want a lightweight and user-friendly inventory solution	Overwhelmed by complicated software
------	--------------------	--------------------------------	---	---	-------------------------------------

Product Inventory Management System Ideation Phase Empathize & Discover

Empathy Map Canvas:

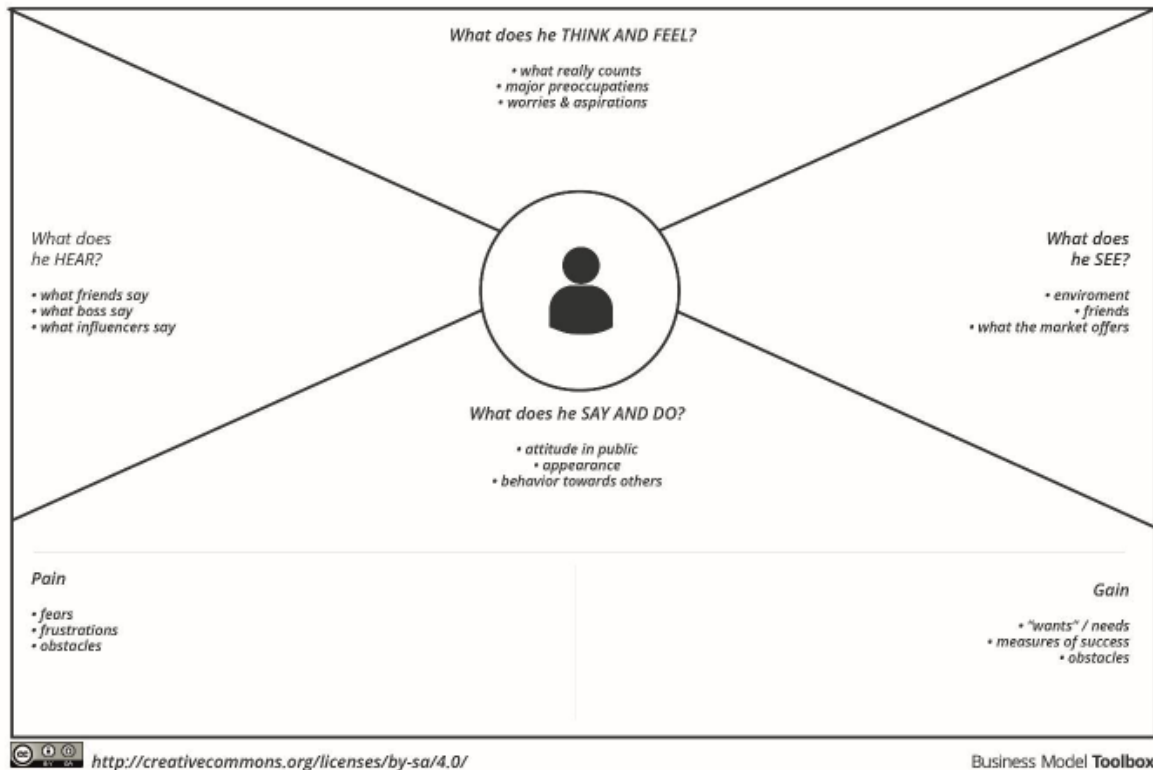
An empathy map is a simple, easy-to-digest visual that captures knowledge about a user's behaviours and attitudes.

It is a useful tool to help teams better understand their users.

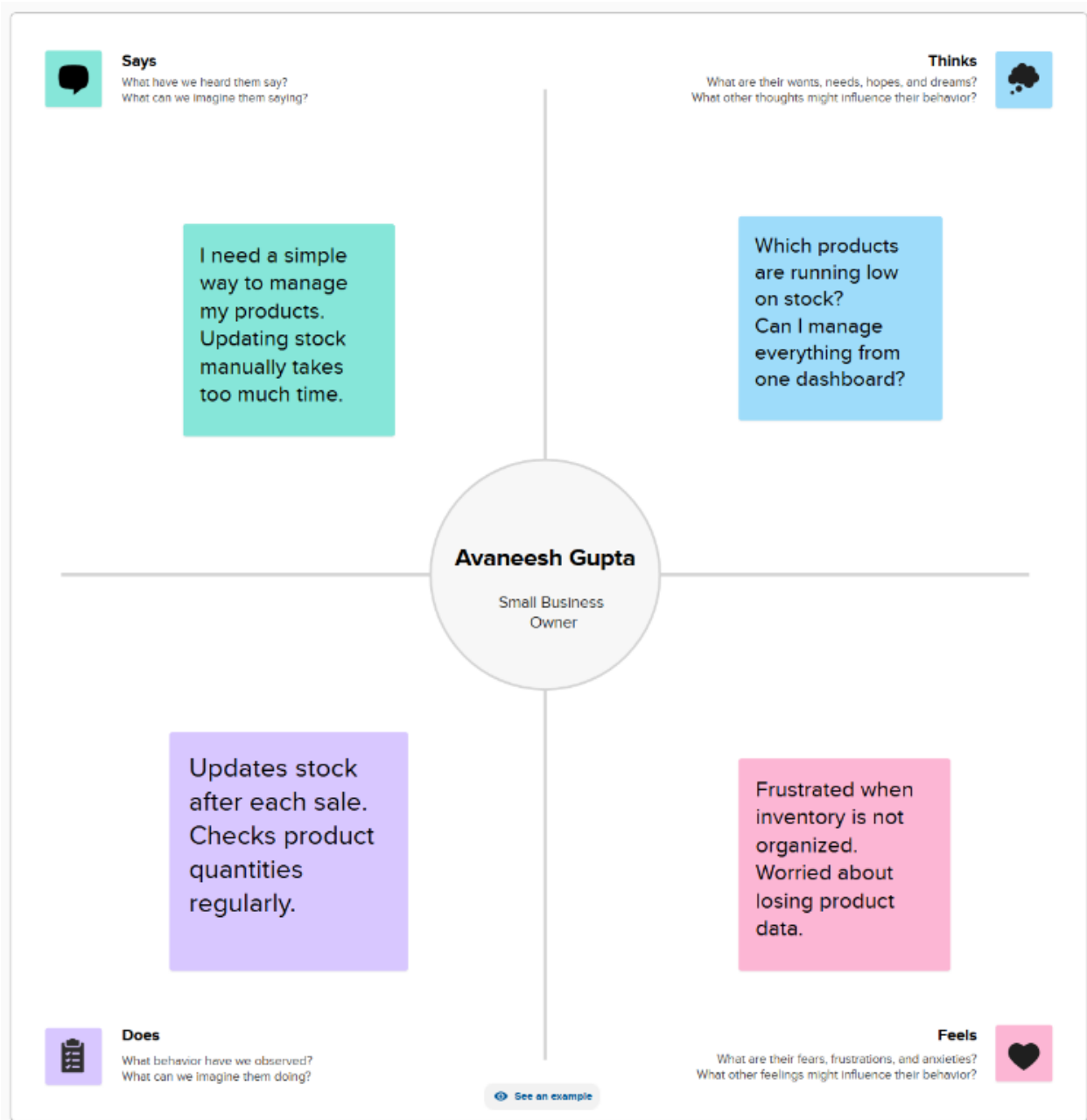
Creating an effective solution requires understanding the true problem and the person who is experiencing it. The exercise of creating the map helps participants consider things from the user's perspective along with his or her goals and challenges.

Example:

Empathy Map



Reference: <https://www.mural.co/templates/empathy-map-canvas>

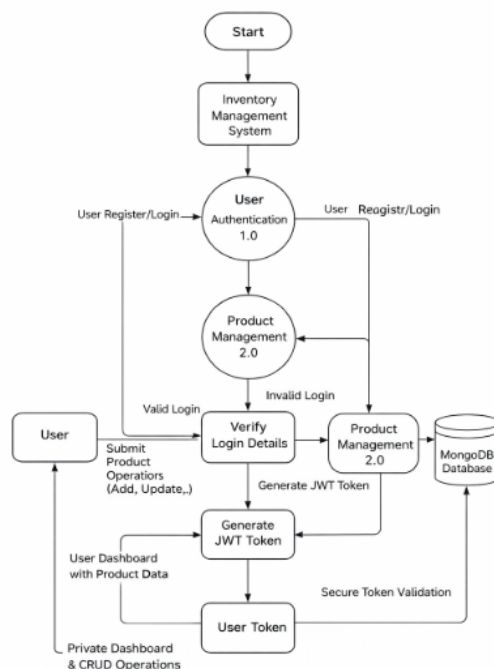


Project Design Phase-II Data Flow Diagram & User Stories

Date	
Team ID	
Project Name	
Maximum Marks	4 Marks

Data Flow Diagrams:

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It shows how data enters and leaves the system, what changes the information, and where data is stored.



User Stories

Use the below template to list all the user stories for the product.

User Type	User Story Number	User Story / Task	Acceptance Criteria	Priority	Release
Customer (Web User)	USN-1	As a user, I can register and log in securely using JWT authentication	User can create account and receive token after login	High	Sprint-1

Customer (Web User)	USN-2	As a user, I can add new products to my inventory	Product is saved and visible only to logged-in user	High	Sprint-1
Customer (Web User)	USN-3	As a user, I can view all my products in a dashboard	Products load successfully with valid token	High	Sprint-1
Customer (Web User)	USN-4	As a user, I can update product details like price and quantity	Changes are saved and reflected instantly	Medium	Sprint-2
Customer (Web User)	USN-5	As a user, I can delete products from my inventory	Product disappears from dashboard after deletion	Medium	Sprint-2
Customer (Web User)	USN-6	As a user, I can search products by name	Filtered results appear based on keyword	Low	Sprint-2

Project Design Phase-II Solution Requirements (Functional & Non-functional)

Functional Requirements:

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	User Authentication & Authorization	User can register with email and password
		User can log in securely using JWT
		Only authenticated users can access dashboard
FR-2	Product Management (CRUD)	Add new product with name, price, quantity and description
		View all products in dashboard

Non-functional Requirements:

Following are the non-functional requirements of the proposed solution.

NFR No.	Non-Functional Requirement	Description
NFR-1	Usability	The application should have a simple and intuitive user interface so users can manage products easily.
NFR-2	Security	JWT authentication must be used. Passwords should be hashed and unauthorized access must be prevented.
NFR-3	Reliability	The system should provide consistent performance with minimal server errors or downtime.
NFR-4	Performance	Product data should load quickly and API responses should be returned within 2 seconds.
NFR-5	Availability	The system should remain accessible to users with minimal downtime during operation.
NFR-6	Scalability	The architecture should support increasing numbers of users and products without performance issues. Future features should be easy to integrate.

Project Design Phase-II Technology Stack (Architecture & Stack)

Technical Architecture:

The system follows a MERN Stack Architecture where the React frontend communicates with a Node.js + Express backend through REST APIs. Authentication is handled using JWT tokens, and all user and product data is stored in MongoDB.

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1	Application Logic-1	User authentication, JWT token generation and validation	Node.js, Express.js, bcryptjs, jsonwebtoken
2	Application Logic-2	Product CRUD operations and search functionality	Node.js, Express.js REST API
3	Database	Store user accounts and product inventory data securely	MongoDB Atlas, Mongoose

4	API Communication	Handles secure data transfer between frontend and backend	REST API, Axios, HTTPS
---	-------------------	---	------------------------

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Backend frameworks used in the system	Node.js, Express.js, Mongoose
2	Scalable Architecture	Modular MVC architecture supporting future features	REST API Architecture, JWT Middleware
3	Security	Secure authentication and protected routes	JWT Authentication, bcrypt Password Hashing
4	Performance Optimization	Fast loading UI and efficient database queries	Vite, MongoDB Indexing
5	Responsive Design	Supports multiple devices and screen sizes	Tailwind CSS, Responsive Layout

Project Planning Phase

Project Planning Template (Product Backlog, Sprint Planning, Stories, Story points)

Product Backlog, Sprint Schedule, and Estimation (4 Marks)

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-1	Backend Setup	USN-1	Initialize Node.js project and configure Express server	6	High	Avaneesh
Sprint-1	Database Setup	USN-2	Connect MongoDB Atlas and create database structure	6	High	Avaneesh

Sprint-1	User Model	USN-3	Create User schema with password hashing	8	High	Avaneesh
Sprint-2	JWT Authentication	USN-4	Implement register and login APIs with JWT token	8	High	Avaneesh
Sprint-2	Auth Middleware	USN-5	Create JWT verification middleware for protected routes	6	High	Avaneesh
Sprint-2	Product Model	USN-6	Design Product schema linked with userId	6	High	Avaneesh
Sprint-3	Product CRUD APIs	USN-7	Implement Add, Get, Update, Delete product APIs	10	High	Avaneesh
Sprint-3	Search API	USN-8	Implement product search/filter endpoint	6	Medium	Avaneesh
Sprint-3	Authorization Logic	USN-9	Ensure users can access only their own products	6	High	Avaneesh
Sprint-4	Error Handling	USN-10	Implement centralized error handling middleware	6	Medium	Avaneesh
Sprint-4	API Testing	USN-11	Test APIs using Postman and validate responses	5	Medium	Avaneesh
Sprint-4	Backend Optimization	USN-12	Improve API performance and validation checks	5	Medium	Avaneesh

Project Tracker, Velocity & Burndown Chart: (4 Marks)

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed	Sprint Release Date (Actual)
Sprint-1	20	5 Days	17 Feb 2026	21 Feb 2026	20	21 Feb 2026

Sprint-2	20	5 Days	22 Feb 2026	26 Feb 2026	20	26 Feb 2026
Sprint-3	22	5 Days	27 Feb 2026	3 Mar 2026	22	3 Mar 2026
Sprint-4	16	4 Days	4 Mar 2026	8 Mar 2026	16	8 Mar 2026

Project Design Phase

Problem – Solution Fit Template

Problem – Solution Fit Overview:

The Problem–Solution Fit ensures that the backend system is designed to solve real-world challenges related to product inventory management. It validates that the proposed API architecture, authentication system, and database design effectively address user needs for secure and efficient inventory handling.

Purpose:

- Address the lack of a secure and structured backend system for managing product inventories.
- Provide reliable REST APIs for product CRUD operations with private user-based access.
- Implement JWT-based authentication to ensure secure communication between clients and server.
- Enable efficient data storage and retrieval using MongoDB with proper authorization checks.

- Improve backend scalability and maintainability using modular Node.js and Express architecture.

Problem Statement:

Many small businesses and individual sellers rely on manual systems or unsecured applications to manage product inventories. Existing solutions often lack proper authentication, structured APIs, or user-specific data protection, which can lead to data inconsistency and unauthorized access. Without a well-designed backend architecture, managing inventory securely and efficiently becomes difficult.

Solution

A Node.js and Express.js based Backend Application designed to provide secure and scalable APIs for product inventory management.

- JWT authentication ensures that only authorized users can access protected routes.
- RESTful APIs allow users to create, read, update, and delete products efficiently.
- MongoDB database integration stores user and product data securely with userId-based authorization.
- Middleware-based architecture handles token verification, request validation, and error handling.
- Modular backend structure improves scalability, maintainability, and future feature expansion

- Project Design Phase
- Proposed Solution Template
- Proposed Solution Template:
- Project team shall fill the following information in the proposed solution template.

S.No.	Parameter	Description
1	Problem Statement (Problem to be solved)	Many small businesses and individual users struggle to manage product inventory securely due to lack of structured backend systems. Existing solutions either lack proper authentication, have complex setups, or do not provide private user-based data management, leading to inefficient inventory handling.
2	Idea / Solution Description	Develop a Node.js and Express.js based backend system that provides secure REST APIs for product inventory management. The solution uses JWT authentication, MongoDB database integration, and middleware-based authorization to ensure users can securely perform CRUD operations on their own products.
3	Novelty / Uniqueness	The backend focuses on user-specific inventory management with JWT-protected APIs, modular MVC architecture, and scalable REST endpoints. It ensures data privacy by linking each product with a unique userId and implementing secure token verification for all protected routes.
4	Social Impact / Customer Satisfaction	The system helps small businesses and individual sellers manage inventory efficiently without expensive software. Secure backend APIs promote data privacy, reliability, and scalability, improving overall user trust and satisfaction.

Project Design Phase Solution Architecture

Solution Architecture:

The solution architecture for the Product Inventory Management System focuses on building a secure, scalable, and maintainable backend using Node.js, Express.js, and MongoDB. The backend provides RESTful APIs for authentication and product management while ensuring data privacy through JWT-based authorization.

Goals of the Solution Architecture:

Identify the Best Tech Solution:

- Use Node.js, Express.js, MongoDB, and JWT authentication to build a secure and efficient backend API for product inventory management.

Define Structure & Characteristics:

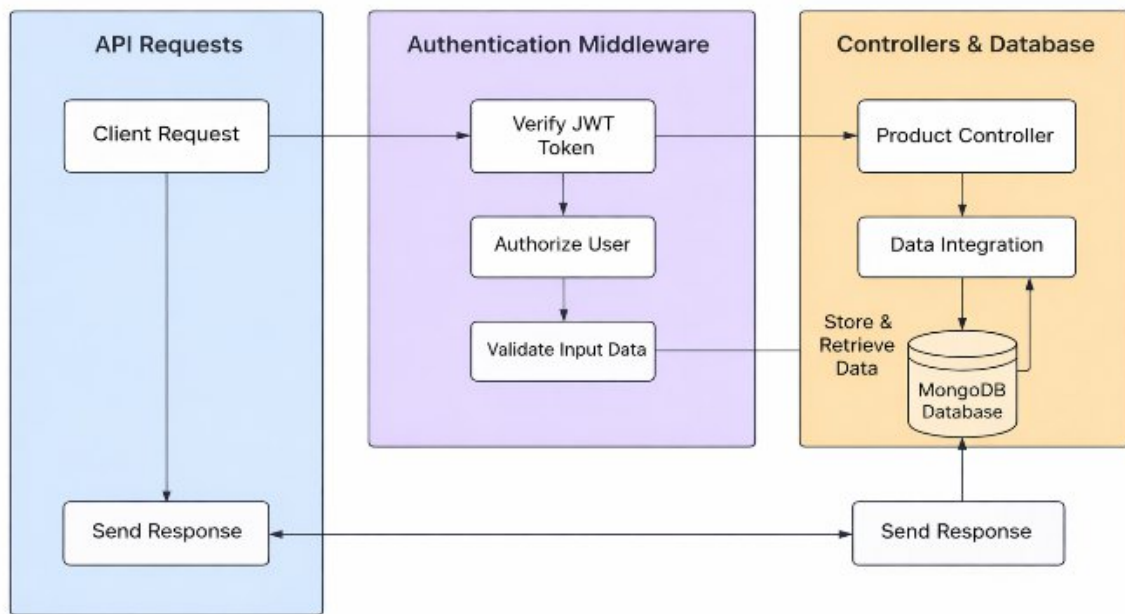
- Follow a modular MVC architecture to ensure scalability, maintainability, and clear separation of authentication and product management logic.

Outline Features & Development Phases:

- Plan development in phases including backend setup, authentication APIs, product CRUD operations, and testing for stable delivery.

Establish Specifications for Development & Delivery:

- Define REST API standards, token-based authorization, and structured middleware to maintain consistent and secure data flow.
- Optimize database queries and asynchronous API handling to provide fast response times and reliable backend performance.



User Acceptance Testing (UAT) Template



Project Overview

Project Name: Product Inventory Management System (Backend)

Project Description: A Node.js and Express.js based backend application that provides secure REST APIs for managing product inventory using JWT authentication. The system allows users to register, log in, and perform CRUD operations on their own products while ensuring data privacy through token-based authorization.

Project Version: v1.0

Testing Period: March 5, 2026 – March 10, 2026

Testing Scope

Features and Functionalities to be Tested

- User Registration API
- User Login API with JWT Token
- Authentication Middleware (Protected Routes)
- Product CRUD APIs (Add, View, Update, Delete)
- Search/Filter API
- Database Integration with MongoDB
- Error Handling & API Response Validation

User Stories or Requirements to be Tested

- User can register and receive valid API response
 - User can log in and receive JWT token
 - Only authenticated users can access product routes
 - Users can manage only their own products
 - API returns proper error messages for invalid requests
-

Test Cases

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Pass/Fail
TC-001	Register User API	1. Send POST request to /register 2. Provide valid data	User created successfully with response message	[Actual Result]	[Pass/Fail]
TC-002	Login API	1. Send POST request to /login 2. Provide credentials	JWT token returned in response	[Actual Result]	[Pass/Fail]
TC-003	Protected Route Access	1. Call /products without token	Access denied with 401 error	[Actual Result]	[Pass/Fail]
TC-004	Create Product API	1. Send POST request with token	Product saved in database	[Actual Result]	[Pass/Fail]
TC-005	Update Product API	1. Send PUT request with token	Product updated successfully	[Actual Result]	[Pass/Fail]
TC-006	Delete Product API	1. Send DELETE request with token	Product removed from database	[Actual Result]	[Pass/Fail]

Bug Tracking

Bug ID	Bug Description	Steps to Reproduce	Severity	Status	Additional Feedback
BG-001	Unauthorized access allowed without token	Call protected route without JWT	High	Open	Fix middleware validation
BG-002	Product update fails with invalid ID	Send incorrect product ID	Medium	In Progress	Add validation check
BG-003	API response delay during large data fetch	Request multiple products	Low	Open	Optimize database query

Sign-off

Tester Name: [Enter Name]

Date: [Enter Date of Completion]

Signature: [Enter Signature]

Notes

- Ensure testing covers both positive & negative cases
- Bug tracking should include severity levels & reproduction steps
- Final sign-off required before deployment

8. Conclusion

A Product Inventory Management System is essential for businesses to manage their stock effectively. It helps in tracking products, reducing errors, and improving overall efficiency. Implementing such a system can greatly enhance business operations and customer satisfaction.